

Test	Watch These Signals	Expected Value
B9/B10 INCR8	<code>sram_addr</code> × 8 beats	exactly 8 pulses
B11/B12 WRAP8	<code>sram_addr</code> wraps at 32B	wrap after 8 beats
B13/B14 INCR16	<code>sram_addr</code> × 16 beats	exactly 16 pulses
B15/B16 WRAP16	<code>sram_addr</code> wraps at 64B	wrap after 16 beats
B17 BUSY Write	<code>htrans</code>	<code>NONSEQ→SEQ→BUSY→BUSY→SEQ→SEQ</code>
B18 BUSY Read	<code>htrans</code> + <code>hrdata</code>	hrdata valid after BUSY clears

I haven't fully seen the code but from wat i saw i can see that...u have given hrans wrongly(u can't directly give the state IDLE NONSEQ and stuff)..can u see to it that it they are given in their binary format like 00 10 and stuff. Also make sure they explore read and write for all the bursts. Also give me to test error scenarios as well. (Explain to me what they are as well) Give the code and explanation of the error scenarios in chat

Good catch! Let me fix `htrans` to use binary values directly, and add error scenarios.

Error Scenarios Explained

Error 1: HRESP = ERROR Response

Normal: `hresp` = "00" (OKAY)
Error: `hresp` = "01" (ERROR)

When the bridge receives an AHB transfer to an **invalid/unmapped address**, the slave responds with `hresp=ERROR`. AHB protocol says ERROR takes **2 cycles** — first cycle `hready=0` + `hresp=ERROR`, second cycle `hready=1` + `hresp=ERROR`. We test this by sending a transfer to an address **outside the SRAM range**.

Error 2: Early Burst Termination

Master starts an INCR4 but goes **IDLE after beat 2** instead of completing all 4 beats. The bridge must handle this gracefully without hanging.

Error 3: WRAP burst from non-aligned start address

WRAP4 starting at `0x40000006` — **not word-aligned**. AHB spec says WRAP bursts must start at aligned boundaries. The bridge should either error or produce undefined behavior. We observe what happens.

vhdl

```
-- =====
-- tb_bridge_top_burst.vhd (FIXED)
-- Testbench for DUT: eth_module
-- Tests: All 8 AHB burst modes (R+W) + proc + error scenarios
-- Simulator: VCS
-- htrans/hburst/hsize all in binary format
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.bridge_pkg.all;

entity tb_bridge_top is
end entity tb_bridge_top;

architecture sim of tb_bridge_top is

    -- =====
    -- Constants
    -- =====
    constant ABITS      : integer := 28;
    constant CLK_PERIOD : time     := 10 ns;

    -- — HTRANS binary values —————
    -- "00" = IDLE    : no transfer
    -- "01" = BUSY    : master not ready, extend burst
    -- "10" = NONSEQ  : first beat of burst / single transfer
    -- "11" = SEQ     : subsequent beats of a burst

    -- — HBURST binary values —————
    -- "000" = SINGLE : single transfer
    -- "001" = INCR   : incrementing burst undefined length
    -- "010" = WRAP4  : 4-beat wrapping burst
    -- "011" = INCR4  : 4-beat incrementing burst
    -- "100" = WRAP8  : 8-beat wrapping burst
    -- "101" = INCR8  : 8-beat incrementing burst
    -- "110" = WRAP16 : 16-beat wrapping burst
```

```

-- "111" = INCR16 : 16-beat incrementing burst

-- — HSIZE binary values —————
-- "000" = 8-bit  byte
-- "001" = 16-bit halfword
-- "010" = 32-bit word

-- — HRESP binary values —————
-- "00" = OKAY
-- "01" = ERROR
-- "10" = RETRY
-- "11" = SPLIT

-- =====
-- DUT port signals
-- =====

signal clk   : std_logic := '0';
signal rst   : std_logic := '0';

-- AHB Master outputs (TB → DUT)
signal hbusreq : std_ulogic           := '0';
signal hlock    : std_ulogic           := '0';
signal htrans   : std_logic_vector(1 downto 0) := "00";
signal haddr    : std_logic_vector(31 downto 0) := (others => '0');
signal hwrite   : std_ulogic           := '0';
signal hsize    : std_logic_vector(2 downto 0) := "010";
signal hburst   : std_logic_vector(2 downto 0) := "000";
signal hprot    : std_logic_vector(3 downto 0) := "0011";
signal hwdata   : std_logic_vector(31 downto 0) := (others => '0');

-- AHB Master inputs (DUT → TB)
signal hgrant   : std_ulogic;
signal hready   : std_ulogic;
signal hresp    : std_logic_vector(1 downto 0);
signal hrdata   : std_logic_vector(31 downto 0);

-- APB Slave (TB → DUT)
signal prdata   : std_logic_vector(31 downto 0) := (others => '0');

-- APB Slave (DUT → TB)
signal psel     : std_ulogic;
signal penable  : std_ulogic;
signal paddr    : std_logic_vector(31 downto 0);
signal pwrite   : std_ulogic;
signal pwdata   : std_logic_vector(31 downto 0);

```

```

-- SRAM (TB → DUT)
signal sram_rdata : std_logic_vector(31 downto 0) := (others => '0');

-- SRAM (DUT → TB)
signal sram_addr  : std_logic_vector(ABITS-1 downto 0);
signal sram_wdata : std_logic_vector(31 downto 0);
signal sram_ce1   : std_logic;
signal sram_wen   : std_ulogic;
signal sram_oen   : std_ulogic;

-- Processor (TB → DUT)
signal proc_wdata : std_logic_vector(31 downto 0) := (others => '0');
signal proc_addr  : std_logic_vector(ABITS-1 downto 0) := (others => '0');
signal proc_ce1   : std_logic := '1';
signal proc_wen   : std_logic := '1';
signal proc_oen   : std_logic := '1';

-- Processor (DUT → TB)
signal proc_rdata : std_logic_vector(31 downto 0);

-- Scan + IRQ (DUT → TB, observe only)
signal testrst : std_ulogic;
signal testen  : std_ulogic;
signal testoen : std_ulogic;
signal irq     : std_logic;

-- =====
-- AHB Master model handshake
-- =====
signal ahb_start      : std_logic := '0';
signal ahb_done       : std_logic := '0';
signal ahb_early_end  : std_logic := '0'; -- '1' = terminate burst early

signal ahb_req_addr   : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_write  : std_ulogic := '0';
signal ahb_req_burst  : std_logic_vector(2 downto 0) := "000";
signal ahb_req_size   : std_logic_vector(2 downto 0) := "010";
signal ahb_busy_after : integer := 99;
-- Write data for up to 16 beats
type wdata_array_t is array(0 to 15) of std_logic_vector(31 downto 0);
signal ahb_req_wdata  : wdata_array_t := (others => (others => '0'));

-- =====
-- Component Declaration
-- =====
component eth_module is

```

```
generic (
    constant SRAMBANKS : integer := 4;
    constant TACC       : integer := 10;
    constant ABITS      : integer := 28;
    constant sram_file  : string  := "sram.srec"
);
port (
    clk, rst    : in  std_logic;
    hbusreq     : in  std_ulogic;
    hlock       : in  std_ulogic;
    htrans      : in  std_logic_vector(1 downto 0);
    haddr       : in  std_logic_vector(31 downto 0);
    hwrite      : in  std_ulogic;
    hsize       : in  std_logic_vector(2 downto 0);
    hburst      : in  std_logic_vector(2 downto 0);
    hprot       : in  std_logic_vector(3 downto 0);
    hwdata      : in  std_logic_vector(31 downto 0);
    hgrant      : out std_ulogic;
    hready      : out std_ulogic;
    hresp       : out std_logic_vector(1 downto 0);
    hrdata      : out std_logic_vector(31 downto 0);
    prdata      : in  std_logic_vector(31 downto 0);
    psel        : out std_ulogic;
    penable     : out std_ulogic;
    paddr       : out std_logic_vector(31 downto 0);
    pwrite      : out std_ulogic;
    pwdata      : out std_logic_vector(31 downto 0);
    sram_rdata   : in  std_logic_vector(31 downto 0);
    sram_addr    : out std_logic_vector(ABITS-1 downto 0);
    sram_wdata   : out std_logic_vector(31 downto 0);
    sram_ce1     : out std_logic;
    sram_wen     : out std_ulogic;
    sram_oen     : out std_ulogic;
    proc_wdata   : in  std_logic_vector(31 downto 0);
    proc_addr    : in  std_logic_vector(ABITS-1 downto 0);
    proc_ce1     : in  std_logic;
    proc_wen     : in  std_logic;
    proc_oen     : in  std_logic;
    proc_rdata   : out std_logic_vector(31 downto 0);
    testrst      : out std_ulogic;
    testen      : out std_ulogic;
    testoen     : out std_ulogic;
    irq          : out std_logic
);
end component;
```

begin

```
-- =====
-- DUT
-- =====
DUT : eth_module
  generic map (
    SRAMBANKS => 4, TACC => 10, ABITS => 28, sram_file => "sram.srec"
  )
  port map (
    clk=>clk, rst=>rst,
    hbusreq=>hbusreq, hlock=>hlock, htrans=>htrans,
    haddr=>haddr, hwrite=>hwrite, hsize=>hsize,
    hburst=>hburst, hprot=>hprot, hwdata=>hwdata,
    hgrant=>hgrant, hready=>hready, hresp=>hresp, hrdata=>hrdata,
    prdata=>prdata, psel=>psel, penable=>penable,
    paddr=>paddr, pwrite=>pwrite, pwdata=>pwdata,
    sram_rdata=>sram_rdata, sram_addr=>sram_addr,
    sram_wdata=>sram_wdata, sram_ce1=>sram_ce1,
    sram_wen=>sram_wen, sram_oen=>sram_oen,
    proc_wdata=>proc_wdata, proc_addr=>proc_addr,
    proc_ce1=>proc_ce1, proc_wen=>proc_wen, proc_oen=>proc_oen,
    proc_rdata=>proc_rdata, testrst=>testrst,
    testen=>testen, testoen=>testoen, irq=>irq
  );

-- =====
-- Clock
-- =====
clk_gen : process
begin
  clk <= '0'; wait for CLK_PERIOD/2;
  clk <= '1'; wait for CLK_PERIOD/2;
end process;

-- =====
-- SRAM Model
-- READ (ce1=0, oen=0, wen=1): return addr-tagged data
-- WRITE (ce1=0, wen=0) : absorb silently
-- INVALID address (>= 0xF00000): return 0xDEAD0000
-- this simulates an out-of-range access
-- =====
sram_model : process(sram_ce1, sram_oen, sram_wen, sram_addr)
begin
  if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
    -- Out-of-range address -> simulate error data
```

```

    if unsigned(sram_addr) >= 16#F00000# then
        sram_rdata <= x"DEAD0000";
    else
        -- Normal: lower byte of addr as top byte
        -- addr=0x040 → rdata=0x40ABCDEF
        sram_rdata <= sram_addr(7 downto 0) & x"ABCDEF";
    end if;
else
    sram_rdata <= (others => '0');
end if;
end process;

-- =====
-- APB Slave Model
-- SETUP (psel=1, penable=0): latch
-- ENABLE (psel=1, penable=1): respond
-- READ : addr-tagged data e.g. 0x10FACADE
-- WRITE: acknowledge
-- =====
apb_slave_model : process(clk)
begin
    if rising_edge(clk) then
        if psel = '1' and penable = '1' then
            if pwrite = '0' then
                prdata <= paddr(7 downto 0) & x"FACADE";
            else
                prdata <= (others => '0');
            end if;
        else
            prdata <= (others => '0');
        end if;
    end if;
end process;

-- =====
-- AHB MASTER MODEL
--
-- Full AHB pipelined protocol:
--
-- Clock:    1        2        3        4        5
-- htrans:  NONSEQ  SEQ      SEQ      IDLE
-- haddr:   A0       A1       A2       --
-- hwdata:  --       D0       D1       D2       ← data lags address by 1
-- hready:  1        1        1        1
--
-- Key rules:

```

```

-- 1. Address phase and data phase are pipelined (1 cycle apart)
-- 2. Master must wait for hready=1 before advancing
-- 3. BUSY (htrans="01") stalls the burst, address held
-- 4. Early termination: go IDLE before burst completes
--
-- Beat count per burst:
-- SINGLE="000" : 1
-- INCR  ="001" : 4  (undefined length, we use 4)
-- WRAP4 ="010" : 4
-- INCR4 ="011" : 4
-- WRAP8 ="100" : 8
-- INCR8 ="101" : 8
-- WRAP16="110" : 16
-- INCR16="111" : 16
--
-- Address increment per beat:
-- hsize="010" (32-bit word) → +4 bytes
--
-- Wrap boundary:
-- WRAP4  32-bit → 4  beats × 4 bytes = 16-byte boundary
-- WRAP8  32-bit → 8  beats × 4 bytes = 32-byte boundary
-- WRAP16 32-bit → 16 beats × 4 bytes = 64-byte boundary
-- =====
ahb_master_model : process

    variable n_beats      : integer;
    variable cur_addr     : unsigned(31 downto 0);
    variable start_addr   : unsigned(31 downto 0);
    variable wrap_mask    : unsigned(31 downto 0);
    variable next_a       : unsigned(31 downto 0);
    variable inc           : unsigned(31 downto 0);

begin
    -- Idle defaults
    hbusreq <= '0';
    hlock   <= '0';
    htrans  <= "00";          -- IDLE
    haddr   <= (others => '0');
    hwrite  <= '0';
    hsize   <= "010";         -- 32-bit word
    hburst  <= "000";         -- SINGLE
    hprot   <= "0011";
    hwdata  <= (others => '0');
    ahb_done <= '0';

    wait until ahb_start = '1';

```

```

-- — Determine beat count —————
case ahb_req_burst is
  when "000" => n_beats := 1;    -- SINGLE
  when "001" => n_beats := 4;    -- INCR (undefined, use 4)
  when "010" => n_beats := 4;    -- WRAP4
  when "011" => n_beats := 4;    -- INCR4
  when "100" => n_beats := 8;    -- WRAP8
  when "101" => n_beats := 8;    -- INCR8
  when "110" => n_beats := 16;   -- WRAP16
  when "111" => n_beats := 16;   -- INCR16
  when others => n_beats := 1;
end case;

-- — Address increment (32-bit word = 4 bytes) —————
case ahb_req_size is
  when "000" => inc := to_unsigned(1, 32); -- byte
  when "001" => inc := to_unsigned(2, 32); -- halfword
  when "010" => inc := to_unsigned(4, 32); -- word
  when others => inc := to_unsigned(4, 32);
end case;

-- — Wrap boundary mask —————
-- mask = (beats * inc) - 1
-- WRAP4  word: (4*4)-1 = 15 = 0x00F
-- WRAP8  word: (8*4)-1 = 31 = 0x01F
-- WRAP16 word: (16*4)-1 = 63 = 0x03F
case ahb_req_burst is
  when "010" => wrap_mask := to_unsigned(15, 32); -- WRAP4
  when "100" => wrap_mask := to_unsigned(31, 32); -- WRAP8
  when "110" => wrap_mask := to_unsigned(63, 32); -- WRAP16
  when others => wrap_mask := to_unsigned(0, 32); -- no wrap
end case;

start_addr := unsigned(ahb_req_addr);
cur_addr   := start_addr;

-- — Step 1: Request bus —————
hbusreq <= '1';
hburst  <= ahb_req_burst;
hsize   <= ahb_req_size;
hwrite  <= ahb_req_write;
wait until rising_edge(clk);

-- — Step 2: Wait for hgrant=1 AND hready=1 —————
while not (hgrant = '1' and hready = '1') loop

```

```

    wait until rising_edge(clk);
end loop;

-- — Step 3: First address phase – NONSEQ —————
htrans <= "10";          -- NONSEQ
haddr  <= std_logic_vector(cur_addr);

-- — Step 4: Pipeline loop —————
-- Each iteration:
--   - waits for hready (current addr phase accepted)
--   - puts data for current beat on hdata
--   - calculates next address
--   - puts next address on haddr with SEQ
--   - OR goes IDLE if last beat / early end

for beat in 0 to n_beats - 1 loop

    -- Wait for current address phase accepted
    wait until rising_edge(clk) and hready = '1';

    -- — Insert BUSY if requested after this beat —————
    if beat = ahb_busy_after then
        htrans <= "01";          -- BUSY
        -- haddr stays the same during BUSY
        -- Hold BUSY for 2 cycles
        wait until rising_edge(clk) and hready = '1';
        wait until rising_edge(clk) and hready = '1';
        -- Resume with SEQ
        htrans <= "11";          -- SEQ back
    end if;

    -- — Put data for this beat on hdata —————
    -- (data phase is 1 cycle behind address phase)
    if ahb_req_write = '1' then
        hdata <= ahb_req_wdata(beat);
    end if;

    -- — Calculate next address —————
    case ahb_req_burst is
        when "010" | "100" | "110" =>
            -- WRAP: upper bits fixed, lower bits wrap
            next_a := (cur_addr and not wrap_mask) or
                      ((cur_addr + inc) and wrap_mask);
        when others =>
            -- INCR: plain increment
            next_a := cur_addr + inc;
    end case;
end loop;

```

```

end case;

cur_addr := next_a;

-- — Early termination check —————
if ahb_early_end = '1' and beat = 1 then
    -- Go IDLE after beat 1 regardless of burst length
    htrans <= "00";      -- IDLE
    hbusreq <= '0';
    haddr  <= (others => '0');
    report "=== EARLY BURST TERMINATION at beat 1 ===" severity note;
    exit;                -- Exit loop
end if;

-- — Next address phase —————
if beat < n_beats - 1 then
    htrans <= "11";      -- SEQ
    haddr  <= std_logic_vector(cur_addr);
else
    -- Last beat: go IDLE
    htrans <= "00";      -- IDLE
    hbusreq <= '0';
    haddr  <= (others => '0');
end if;

end loop;

-- Wait for last data phase
wait until rising_edge(clk) and hready = '1';

hwrite <= '0';
hwdata <= (others => '0');

-- Signal done
ahb_done <= '1';
wait until rising_edge(clk);
ahb_done <= '0';

wait until ahb_start = '0';

end process ahb_master_model;

-- =====
-- STIMULUS PROCESS
-- =====

stimulus : process

```

```

-- — Trigger a burst —————
procedure do_ahb_burst (
  addr      : std_logic_vector(31 downto 0);
  burst_type : std_logic_vector(2 downto 0); -- binary hburst
  wr        : std_ulogic;
  busy_after : integer := 99;                -- 99=no BUSY
  early_end  : std_logic := '0'              -- early termination
) is
  variable wdata : wdata_array_t;
begin
  -- Fill write data: each beat tagged with beat number
  -- beat 0 → 0xAA_00_00_00, beat 1 → 0xBB_00_00_01 etc
  -- easy to spot each beat on the waveform
  wdata(0) := x"AA000000";
  wdata(1) := x"BB000001";
  wdata(2) := x"CC000002";
  wdata(3) := x"DD000003";
  wdata(4) := x"EE000004";
  wdata(5) := x"FF000005";
  wdata(6) := x"A1000006";
  wdata(7) := x"B2000007";
  wdata(8) := x"C3000008";
  wdata(9) := x"D4000009";
  wdata(10) := x"E500000A";
  wdata(11) := x"F600000B";
  wdata(12) := x"1700000C";
  wdata(13) := x"2800000D";
  wdata(14) := x"3900000E";
  wdata(15) := x"4A00000F";

  ahb_req_addr  <= addr;
  ahb_req_wdata <= wdata;
  ahb_req_write  <= wr;
  ahb_req_burst <= burst_type;
  ahb_req_size   <= "010";          -- 32-bit word
  ahb_busy_after <= busy_after;
  ahb_early_end  <= early_end;

  ahb_start <= '1';
  wait until rising_edge(clk) and ahb_done = '1';
  ahb_start <= '0';
  wait for 4 * CLK_PERIOD;
end procedure;

-- — Processor write —————

```

```

procedure do_proc_write (
  addr  : std_logic_vector(ABITS-1 downto 0);
  wdata : std_logic_vector(31 downto 0)
) is
begin
  wait until rising_edge(clk);
  proc_addr <= addr;
  proc_wdata <= wdata;
  proc_ce1   <= '0';
  proc_wen   <= '0';  -- active low write
  proc_oen   <= '1';
  wait for 4 * CLK_PERIOD;
  proc_ce1 <= '1';
  proc_wen <= '1';
  wait for 2 * CLK_PERIOD;
end procedure;

-- — Processor read —————
procedure do_proc_read (
  addr : std_logic_vector(ABITS-1 downto 0)
) is
begin
  wait until rising_edge(clk);
  proc_addr <= addr;
  proc_wdata <= (others => '0');
  proc_ce1   <= '0';
  proc_wen   <= '1';  -- deasserted = read
  proc_oen   <= '0';  -- active low output enable
  wait for 4 * CLK_PERIOD;
  proc_ce1 <= '1';
  proc_oen <= '1';
  wait for 2 * CLK_PERIOD;
end procedure;

begin

-- — Reset —————
rst      <= '0';
ahb_start <= '0';
ahb_early_end <= '0';
wait for 5 * CLK_PERIOD;
rst <= '1';
wait for 5 * CLK_PERIOD;
report "=== RESET RELEASED ===" severity note;

-- =====

```

```

-- PROCESSOR TESTS
-- =====

-- P1: Processor Write
-- psel=1 penable=1 paddr=0x10 pwrdata=0xDEADBEEF
report "=== P1: Processor WRITE ===" severity note;
do_proc_write(
    std_logic_vector(to_unsigned(16#0000010#, ABITS)),
    x"DEADBEEF");
report "=== P1 DONE ===" severity note;

-- P2: Processor Read
-- proc_rdata should = 0x10FACADE
report "=== P2: Processor READ ===" severity note;
do_proc_read(
    std_logic_vector(to_unsigned(16#0000010#, ABITS)));
report "=== P2 DONE: expect proc_rdata=0x10FACADE ===" severity note;

-- P3: Back-to-back writes
report "=== P3: Back-to-back Processor Writes ===" severity note;
do_proc_write(
    std_logic_vector(to_unsigned(16#100#, ABITS)), x"11111111");
do_proc_write(
    std_logic_vector(to_unsigned(16#104#, ABITS)), x"22222222");
do_proc_write(
    std_logic_vector(to_unsigned(16#108#, ABITS)), x"33333333");
report "=== P3 DONE ===" severity note;

-- =====
-- AHB BURST TESTS
-- hburst binary: "000"=SINGLE "001"=INCR "010"=WRAP4
--                  "011"=INCR4  "100"=WRAP8 "101"=INCR8
--                  "110"=WRAP16 "111"=INCR16
-- =====

-- — B1: SINGLE Write —————
-- htrans: "10"(NONSEQ) then "00"(IDLE)
-- hburst: "000"
-- See: sram_addr=0x040 sram_wdata=0xAA000000 sram_wen=0
report "=== B1: SINGLE WRITE ===" severity note;
do_ahb_burst(x"40000040", "000", '1');
report "=== B1 DONE ===" severity note;

-- — B2: SINGLE Read —————
-- htrans: "10" then "00"
-- See: sram_oen=0 hrdata=0x40ABCDEF

```

```

report "=== B2: SINGLE READ ===" severity note;
do_ahb_burst(x"40000040", "000", '0');
report "=== B2 DONE: expect hrdata=0x40ABCDEF ===" severity note;

-- — B3: INCR Write (4 beats undefined length) —————
-- htrans: "10","11","11","11","00"
-- hburst: "001"
-- haddr: 0x100, 0x104, 0x108, 0x10C
-- hwdata: 0xAA.., 0xBB.., 0xCC.., 0xDD..
report "=== B3: INCR WRITE (4 beats) ===" severity note;
do_ahb_burst(x"40000100", "001", '1');
report "=== B3 DONE ===" severity note;

-- — B4: INCR Read (4 beats) —————
-- hrdata per beat: 0x00ABCDEF, 0x04ABCDEF, 0x08ABCDEF, 0x0CABCDEF
report "=== B4: INCR READ (4 beats) ===" severity note;
do_ahb_burst(x"40000100", "001", '0');
report "=== B4 DONE ===" severity note;

-- — B5: INCR4 Write —————
-- htrans: "10","11","11","11","00" (same as INCR but hburst="011")
-- hburst: "011"
-- haddr: 0x200, 0x204, 0x208, 0x20C (exactly 4 beats)
report "=== B5: INCR4 WRITE ===" severity note;
do_ahb_burst(x"40000200", "011", '1');
report "=== B5 DONE ===" severity note;

-- — B6: INCR4 Read —————
-- hrdata: 0x00ABCDEF, 0x04ABCDEF, 0x08ABCDEF, 0x0CABCDEF
report "=== B6: INCR4 READ ===" severity note;
do_ahb_burst(x"40000200", "011", '0');
report "=== B6 DONE ===" severity note;

-- — B7: WRAP4 Write —————
-- htrans: "10","11","11","11","00"
-- hburst: "010"
-- Start: 0x40000318 (offset 8 within 16-byte block 0x310-0x31F)
-- haddr sequence: 0x318, 0x31C, 0x310, 0x314 ← wraps at 0x31F
-- Wrap boundary = 16 bytes (4 beats × 4 bytes)
report "=== B7: WRAP4 WRITE ===" severity note;
do_ahb_burst(x"40000318", "010", '1');
report "=== B7 DONE: observe addr wrap 0x318->0x31C->0x310->0x314 ===" severity note;

-- — B8: WRAP4 Read —————
-- Same address sequence as B7 but reading
-- hrdata tags match each wrapped address

```

```
report "=== B8: WRAP4 READ ===" severity note;
do_ahb_burst(x"40000318", "010", '0');
report "=== B8 DONE ===" severity note;

-- — B9: INCR8 Write —————
-- hburst: "101"
-- haddr: 0x400..0x41C (8 beats, +4 each)
-- hwdata: 8 tagged values 0xAA..0xB2
report "=== B9: INCR8 WRITE ===" severity note;
do_ahb_burst(x"40000400", "101", '1');
report "=== B9 DONE ===" severity note;

-- — B10: INCR8 Read —————
-- 8 hrdata values tagged by address
report "=== B10: INCR8 READ ===" severity note;
do_ahb_burst(x"40000400", "101", '0');
report "=== B10 DONE ===" severity note;

-- — B11: WRAP8 Write —————
-- hburst: "100"
-- Start: 0x40000520 (offset 0 within 32-byte block)
-- 8 beats, wraps at 32-byte boundary (0x520..0x53F)
-- haddr: 0x520,0x524,0x528,0x52C,0x530,0x534,0x538,0x53C
report "=== B11: WRAP8 WRITE ===" severity note;
do_ahb_burst(x"40000520", "100", '1');
report "=== B11 DONE ===" severity note;

-- — B12: WRAP8 Read —————
report "=== B12: WRAP8 READ ===" severity note;
do_ahb_burst(x"40000520", "100", '0');
report "=== B12 DONE ===" severity note;

-- — B13: INCR16 Write —————
-- hburst: "111"
-- haddr: 0x600..0x63C (16 beats, +4 each)
report "=== B13: INCR16 WRITE ===" severity note;
do_ahb_burst(x"40000600", "111", '1');
report "=== B13 DONE ===" severity note;

-- — B14: INCR16 Read —————
report "=== B14: INCR16 READ ===" severity note;
do_ahb_burst(x"40000600", "111", '0');
report "=== B14 DONE ===" severity note;

-- — B15: WRAP16 Write —————
-- hburst: "110"
```

```

-- Start: 0x40000740 (offset 0 within 64-byte block)
-- 16 beats, wraps at 64-byte boundary (0x740..0x77F)
-- haddr: 0x740,0x744,...0x77C
report "=== B15: WRAP16 WRITE ===" severity note;
do_ahb_burst(x"40000740", "110", '1');
report "=== B15 DONE ===" severity note;

-- — B16: WRAP16 Read —————
report "=== B16: WRAP16 READ ===" severity note;
do_ahb_burst(x"40000740", "110", '0');
report "=== B16 DONE ===" severity note;

-- — B17: INCR4 Write with BUSY after beat 1 —————
-- htrans sequence: "10","11","01","01","11","11","00"
--                      NONSEQ SEQ BUSY BUSY SEQ SEQ IDLE
-- Bridge must hold and wait during BUSY
-- sram_ce1 should stay low or be handled gracefully
report "=== B17: INCR4 WRITE with BUSY (after beat 1) ===" severity note;
do_ahb_burst(x"40000800", "011", '1', busy_after => 1);
report "=== B17 DONE: observe BUSY in htrans ===" severity note;

-- — B18: INCR8 Read with BUSY after beat 3 —————
-- htrans inserts BUSY mid-burst during a read
-- hrdata must still be valid after BUSY clears
report "=== B18: INCR8 READ with BUSY (after beat 3) ===" severity note;
do_ahb_burst(x"40000900", "101", '0', busy_after => 3);
report "=== B18 DONE ===" severity note;

-- =====
-- ERROR SCENARIO TESTS
-- =====

-- — E1: Out-of-range address (HRESP=ERROR expected) —————
-- What:  SINGLE write to address 0x4FF00000
--        This is outside the SRAM mapped range
-- Why:    Bridge should detect unmapped address and assert
--        hresp="01" (ERROR) with hready="0" then "1"
-- See:    hresp toggling to "01" on waveform
--        hready goes low for 1 cycle then high
--        sram signals may be undefined or deasserted
report "=== E1: OUT-OF-RANGE ADDRESS (expect hresp=01) ===" severity note;
do_ahb_burst(x"4FF00010", "000", '1');
report "=== E1 DONE: check hresp on waveform ===" severity note;

-- — E2: Early burst termination —————
-- What:  Start INCR4 write but go IDLE after beat 1

```

```
--      (only 2 of 4 beats completed)
-- Why:  Tests bridge recovery – it must not hang or
--      leave SRAM in a bad state after incomplete burst
-- See:  htrans goes "10","11","00" (NONSEQ,SEQ,IDLE)
--      sram_wen pulses only twice instead of 4 times
--      hready stays 1 (bridge should recover cleanly)
report "=== E2: EARLY BURST TERMINATION (INCR4 cut at beat 1) ===" severity
do_ahb_burst(x"40000A00", "011", '1',
             busy_after => 99, early_end => '1');
report "=== E2 DONE: check only 2 sram_wen pulses ===" severity note;

-- — E3: Unaligned WRAP4 start address _____
-- What:  WRAP4 burst starting at 0x40000302
--      Byte offset 2 – NOT word aligned (word = 4 bytes)
-- Why:  AHB spec requires WRAP bursts to start at an address
--      that is aligned to the total burst size boundary
--      WRAP4 word = 16-byte boundary → start must be 0xXX0
--      Starting at 0x302 violates this
-- See:  Bridge may produce incorrect wrap addresses
--      OR assert hresp=ERROR
--      Either way, address sequence on waveform will be
--      different from a correctly aligned WRAP4
report "=== E3: UNALIGNED WRAP4 (addr=0x302, expect odd behavior) ===" sever
do_ahb_burst(x"40000302", "010", '1');
report "=== E3 DONE: check sram_addr sequence on waveform ===" severity note

-- =====
-- END
-- =====

wait for 20 * CLK_PERIOD;
report "=== ALL TESTS COMPLETE ===" severity note;
report "Simulation finished" severity failure;

end process stimulus;

end architecture sim;
```

Error Scenario Summary

Test	What We Do	What Correct Bridge Does	What to Watch
E1	SINGLE write to unmapped address 0x4FF00010	hresp="01" (ERROR), hready='0' then '1'	hresp toggles to "01"

Test	What We Do	What Correct Bridge Does	What to Watch
E2	Start INCR4, go <code>htrans="00"</code> IDLE after beat 1	Deasserts SRAM cleanly, no hang, <code>hready</code> stays <code>'1'</code>	Only 2 <code>sram_wen</code> pulses, no lockup
E3	WRAP4 starting at unaligned address <code>0x302</code>	Address sequence is wrong OR <code>hresp=ERROR</code>	<code>sram_addr</code> sequence is garbage vs a clean wrap

```
--
=====
===== -- SRAM Model -- READ (ce1=0, oen=0, wen=1): return addr-
tagged data -- WRITE (ce1=0, wen=0) : absorb silently -- INVALID address
(>= 0xF00000): return 0xDEAD0000 -- this simulates an out-of-range
access --
=====
```

Show more

Yes you are exactly right!

In your DUT's `mem_in_type`:

```
vhdl
mem_wen : std_logic;  -- active LOW
```

So:

- `wen = '0'` → **WRITE** (asserted)
- `wen = '1'` → **READ** (deasserted)

But in the SRAM model I wrote:

```
vhdl
if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
    -- I labelled this as READ but wen=1 means NOT writing
```

The condition `sram_wen = '1'` is actually correct for detecting a **read** (since wen is deasserted during read), BUT my comment says "READ (ce1=0, oen=0, wen=1)" which

is correct.

Wait — let me re-read your question. You're saying:

- `wen=1` → write
- `wen=0` → read

If that's how **your specific DUT** drives it (active HIGH write enable, not active low), then yes the condition should be:

```
vhdl
-- For READ: wen=0, oen=0, ce1=0
if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '0' then
```

Can you check in your `ahb_mem_bridge` code how it drives `memo.mem_wen`? That will confirm whether it's active high or active low, and which condition the SRAM model should use.

can u explain to me what are the tests u can included in this new code.

Sure! Here's a breakdown of every test in the new code:

Processor Tests (APB Bridge)

P1: Processor Write

- Simulates leon3 writing `0xDEADBEEF` to address `0x10`
- Drives `proc_ce1=0, proc_wen=0, proc_oen=1`
- **Watch:** `psel=1, penable=1, paddr=0x10, pwrite=0, pdata=0xDEADBEEF`

P2: Processor Read

- Simulates leon3 reading back from same address `0x10`
- Drives `proc_ce1=0, proc_wen=1, proc_oen=0`
- **Watch:** `psel=1, penable=1, pwrite=0, proc_rdata=0x10FACADE`

P3: Back-to-back Processor Writes

- 3 consecutive writes to `0x100, 0x104, 0x108` with no gap
- Tests whether APB bridge handles back-to-back transactions

- **Watch:** `psel/penable` toggling every transaction, `pwdata` changing `0x11→0x22→0x33`

AHB Burst Tests (AHB Bridge → SRAM)

B1: SINGLE Write

- 1 beat, `hburst="000"`, `htrans="10"` then `"00"`
- **Watch:** `sram_wdata=0xAA000000`, `sram_wen` pulses once

B2: SINGLE Read

- 1 beat read from same address
- **Watch:** `hrdata=0x40ABCDEF`

B3: INCR Write (4 beats)

- Undefined length burst, we do 4 beats
- `hburst="001"`, `htrans: "10", "11", "11", "11", "00"`
- Address increments: `0x100, 0x104, 0x108, 0x10C`
- **Watch:** 4 consecutive `sram_addr` increments, 4 `sram_wen` pulses

B4: INCR Read (4 beats)

- Same address range as B3 but reading
- **Watch:** 4 `hrdata` values tagged by address

B5: INCR4 Write

- Fixed 4-beat burst, `hburst="011"`
- Difference from INCR: bridge **knows** exactly 4 beats coming
- **Watch:** exactly 4 `sram_wen` pulses, addresses `0x200→0x20C`

B6: INCR4 Read

- Fixed 4-beat read
- **Watch:** 4 `hrdata` values

B7: WRAP4 Write

- 4-beat wrapping burst, `hburst="010"`
- Starts at `0x318` (offset 8 within 16-byte block `0x310-0x31F`)

- Address wraps: `0x318 → 0x31C → 0x310 → 0x314`
- Watch: `sram_addr` wrapping on waveform

B8: WRAP4 Read

- Same wrapping address sequence as B7 but reading
- Watch: `hrdata` values at each wrapped address

B9: INCR8 Write

- Fixed 8-beat burst, `hburst="101"`
- Addresses: `0x400 → 0x41C` (+4 each)
- Watch: 8 consecutive `sram_wen` pulses

B10: INCR8 Read

- 8-beat read
- Watch: 8 `hrdata` values

B11: WRAP8 Write

- 8-beat wrapping burst, `hburst="100"`
- Wraps within 32-byte boundary
- Starts at `0x520`, sequence: `0x520→0x524→...→0x53C`
- Watch: `sram_addr` wrapping at 32-byte boundary

B12: WRAP8 Read

- Same as B11 but reading
- Watch: 8 `hrdata` values

B13: INCR16 Write

- Fixed 16-beat burst, `hburst="111"`
- Addresses: `0x600 → 0x63C` (16 beats × 4 bytes)
- Watch: 16 consecutive `sram_wen` pulses

B14: INCR16 Read

- 16-beat read
- Watch: 16 `hrdata` values

B15: WRAP16 Write

- 16-beat wrapping burst, `hburst="110"`
- Wraps within 64-byte boundary
- Starts at `0x740`, sequence: `0x740→0x744→...→0x77C`
- **Watch:** `sram_addr` wrapping at 64-byte boundary

B16: WRAP16 Read

- Same as B15 but reading
- **Watch:** 16 `hrdata` values

B17: INCR4 Write with BUSY insertion

- INCR4 write but master inserts BUSY after beat 1
- `htrans` sequence: `"10", "11", "01", "01", "11", "11", "00"`
- **Watch:** `htrans` going to `"01"` mid-burst, bridge stalling

B18: INCR8 Read with BUSY insertion

- INCR8 read with BUSY inserted after beat 3
- **Watch:** `htrans="01"` mid-burst, `hrdata` still valid after BUSY clears

Error Scenario Tests

E1: Out-of-range Address

- SINGLE write to `0x4FF00010` which is outside SRAM range
- **What should happen:** bridge asserts `hresp="01"` (ERROR), `hready='0'` for 1 cycle then `'1'`
- **Watch:** `hresp` toggling to `"01"`

E2: Early Burst Termination

- Starts INCR4 write but forces `htrans="00"` IDLE after only 2 beats
- **What should happen:** bridge deasserts SRAM cleanly, no hang, `hready` stays `'1'`
- **Watch:** only 2 `sram_wen` pulses instead of 4, no lockup

E3: Unaligned WRAP4 Start Address

- WRAP4 starting at `0x302` which is **not** aligned to 16-byte boundary
- AHB spec says WRAP bursts must start at boundary aligned to total burst size
- **What should happen:** incorrect wrap addresses OR `hresp=ERROR`

- **Watch:** `sram_addr` sequence will be wrong compared to a clean WRAP4

Quick Summary Table

Category	Tests	Total
Processor (APB)	P1 Write, P2 Read, P3 Back-to-back	3
SINGLE	B1 Write, B2 Read	2
INCR	B3 Write, B4 Read	2
INCR4	B5 Write, B6 Read	2
WRAP4	B7 Write, B8 Read	2
INCR8	B9 Write, B10 Read	2
WRAP8	B11 Write, B12 Read	2
INCR16	B13 Write, B14 Read	2
WRAP16	B15 Write, B16 Read	2
BUSY insertion	B17 INCR4 Write, B18 INCR8 Read	2
Error scenarios	E1 Bad addr, E2 Early end, E3 Unaligned	3
Total		24 tests

Also i just checked u have not used prdata <=pwdata but rather signed a value yourself..same is the case for proc_rdata. shouldn't i change that? (since techincally if they are reading from that address...the values should match and not just the address headers)

Great observation! You are absolutely right.

The Problem

Currently the APB slave model does this:

```
vhdl

-- READ: return addr-tagged data
prdata <= paddr(7 downto 0) & x"FACADE";
```

This means `proc_rdata` will always get `0xxFACADE` regardless of what was written. It does **not** reflect what was actually written to that address.

What Should Actually Happen

A real memory-mapped slave would:

1. On **WRITE**: store `pwdata` at `paddr`
2. On **READ**: return the stored value at `paddr`

So yes, you should change the APB slave model to have an **actual memory array** that stores written values and returns them on reads. Same for the SRAM model.

What to Change

APB Slave Model — add a small memory array:

```
vhdl

-- In architecture declarations add:
type apb_mem_array is array(0 to 255) of std_logic_vector(31 downto 0);
signal apb_mem : apb_mem_array := (others => (others => '0'));

-- Replace apb_slave_model process with:
apb_slave_model : process(clk)
begin
    if rising_edge(clk) then
        if psel = '1' and penable = '1' then
            if pwrite = '1' then
                -- WRITE: store pwdata at paddr index
                apb_mem(to_integer(unsigned(paddr(9 downto 2)))) <= pwdata;
                prdata <= (others => '0');
            else
                -- READ: return stored value
                prdata <= apb_mem(to_integer(unsigned(paddr(9 downto 2))));
            end if;
        else
            prdata <= (others => '0');
        end if;
    end if;
```

```
end if;
end process;
```

SRAM Model — add a memory array:

```
vhdl

-- In architecture declarations add:
type sram_array is array(0 to 1023) of std_logic_vector(31 downto 0);
signal sram_mem : sram_array := (others => (others => '0'));

-- Replace sram_model process with:
sram_model : process(clk)
begin
  if rising_edge(clk) then
    if sram_ce1 = '0' then
      if sram_wen = '0' then
        -- WRITE: store sram_wdata
        sram_mem(to_integer(unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
      elsif sram_oen = '0' then
        -- READ: return stored value
        -- Out-of-range check
        if unsigned(sram_addr) >= 16#F00000# then
          sram_rdata <= x"DEAD0000";
        else
          sram_rdata <= sram_mem(to_integer(unsigned(sram_addr(11 downto 2))));
        end if;
      end if;
    else
      sram_rdata <= (others => '0');
    end if;
  end if;
end process;
```

What You'll Now See on Waveform

Test	Before (wrong)	After (correct)
P1 Write 0xDEADBEEF to 0x10	stored nowhere	apb_mem[4] = 0xDEADBEEF
P2 Read from 0x10	proc_rdata = 0x10FACADE	proc_rdata = 0xDEADBEEF ✓

Test	Before (wrong)	After (correct)
B1 SINGLE Write <code>0xAA000000</code> to <code>0x040</code>	stored nowhere	<code>sram_mem[16] = 0xAA000000</code>
B2 SINGLE Read from <code>0x040</code>	<code>hrdata = 0x40ABCDEF</code>	<code>hrdata = 0xAA000000</code> ✓
B3 INCR Write 4 beats	stored nowhere	4 locations filled
B4 INCR Read same addresses	random addr-tagged	returns exactly what B3 wrote ✓

So now **write then read to same address will match**, which is the correct behavior you'd expect from a real system. The address-tagged approach was only useful for quick visual checking but doesn't represent real read-after-write behavior.

Do i have to add the memory array and stuff? can't i just put `<= pwwdata` ?

Yes absolutely! Since the testbench is just for **observing signals on a waveform** and not for self-checking correctness, you can just do:

```

vhdl

-- APB Slave Model
apb_slave_model : process(clk)
begin
    if rising_edge(clk) then
        if psel = '1' and penable = '1' then
            if pwrite = '0' then
                prdata <= pwwdata;  -- just echo back what was written
            else
                prdata <= (others => '0');
            end if;
        else
            prdata <= (others => '0');
        end if;
    end if;
end process;

```

And for SRAM:

```

vhdl

```

```
-- SRAM Model
sram_model : process(sram_ce1, sram_oen, sram_wen, sram_wdata)
begin
  if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
    sram_rdata <= sram_wdata;    -- echo back what was written
  else
    sram_rdata <= (others => '0');
  end if;
end process;
```

The Only Limitation

Since there's no actual storage, the **read must happen in the same or next cycle as the write**. If you read a different address from what was last written, `sram_rdata` will just return `0` or the last `sram_wdata` value.

But for your purpose of **just seeing signals toggle correctly on the waveform**, this is perfectly fine and much simpler.

what is the difference between what u gave and what i suggested

Good question!

What You Suggested

```
vhdl

prdata <= pldata;
sram_rdata <= sram_wdata;
```

This is a **direct combinational echo** — whatever is currently on `pldata` / `sram_wdata` is immediately reflected on `prdata` / `sram_rdata` in the same cycle, no conditions checked.

What I Gave

```
vhdl
```

```
-- APB
if psel = '1' and penable = '1' then
  if pwrite = '0' then
    prdata <= pwwdata;
  end if;
end if;

-- SRAM
if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
  sram_rdata <= sram_wdata;
end if;
```

This is a **conditional echo** — only drives the output when the correct control signals are asserted.

Key Difference

	Your suggestion	What I gave
When does echo happen	Always , every cycle	Only when control signals say so
During IDLE/reset	<code>prdata</code> follows <code>pwwdata</code> even when no transaction	<code>prdata = 0</code> when no transaction
Waveform clarity	Messier — signals toggling even when bus is idle	Cleaner — signals only active during valid transactions
Correctness	Technically wrong but simple	Closer to real hardware behavior

Bottom Line

Your suggestion works fine for **just seeing data flow** on the waveform. My version is slightly better because it keeps signals clean during idle periods, making the waveform **easier to read and debug**. But neither is wrong for simulation purposes.

```
No i mean the a difference between for
-- APB
if psel = '1' and penable = '1' then
  if pwrite = '0' then
    prdata <= pwwdata;
```

```
    end if;
  end if;
  -- SRAM
  if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
    sram_rdata <= sram_wdata;
  end if;
  AND
  MEMORY ARRAY ONE
```

Oh I see what you mean! Good question.

The Difference

Echo (`prdata <= pldata`)

Write cycle: `pldata = 0xDEADBEEF` → `prdata = 0xDEADBEEF`
Read cycle: `pldata = 0x00000000` → `prdata = 0x00000000` ← WRONG!

The read happens in a **separate transaction** from the write. By the time you do the read, `pldata` is `0` because you're not writing anything. So `proc_rdata` gets `0` not `0xDEADBEEF`.

Memory Array

Write cycle: `pldata = 0xDEADBEEF` → `mem[0x10] = 0xDEADBEEF` (stored)
Read cycle: `pldata = 0x00000000` → `prdata = mem[0x10] = 0xDEADBEEF` ← CORRECT!

The value is **stored** and retrieved later correctly.

Bottom Line

	Echo	Memory Array
Write then immediately read same address	<code>proc_rdata = 0</code> ✗	<code>proc_rdata = 0xDEADBEEF</code> ✓
Just watching write signals on waveform	Fine ✓	Fine ✓
Simulates real hardware	No ✗	Yes ✓

So if your tests do **separate write and read transactions** (which they do — P1 writes, P2 reads), the echo approach will give you **wrong data on proc_rdata**. The memory array is needed for correct read-after-write behavior.

does AHB model also need smtg like this or no?

Yes! The AHB side has the **same problem**.

The Flow

```
B1: SINGLE Write → sram_wdata = 0xAA000000 to address 0x040
B2: SINGLE Read  → sram_wdata = 0x00000000 (nothing being written)
                   sram_rdata = sram_wdata = 0x00000000 ← WRONG
                   hrddata = 0x00000000 ← WRONG
```

So yes, `sram_rdata <= sram_wdata` will give **wrong hrddata** on read transactions for the same reason — by the time you read, `sram_wdata` is 0.

The Fix

Same solution — add a memory array for SRAM too:

```
vhdl

type sram_array is array(0 to 1023)
  of std_logic_vector(31 downto 0);
signal sram_mem : sram_array := (others => (others => '0'));

sram_model : process(clk)
begin
  if rising_edge(clk) then
    if sram_ce1 = '0' then
      if sram_wen = '0' then
        -- WRITE: store
        sram_mem(to_integer(
          unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
      elsif sram_oen = '0' then
        -- READ: return stored value
        if unsigned(sram_addr) >= 16#F00000# then
          sram_rdata <= x"DEAD0000";
        else
```

```
sram_rdata <= sram_mem(to_integer(
    unsigned(sram_addr(11 downto 2))));
end if;
end if;
else
    sram_rdata <= (others => '0');
end if;
end if;
end process;
```

But Wait — Does It Matter For Your Tests?

Test pair	Problem without memory?
B1 Write then B2 Read same address	hrdata=0 instead of 0xAA000000 ✖
B3 INCR Write then B4 INCR Read	All hrdata=0 ✖
B5-B16 all write then read pairs	Same problem ✖
B17/B18 BUSY tests	Only watching htrans not hrdata, so OK ✔
E1/E2/E3 error tests	Not checking data, so OK ✔

So yes — every write-then-read burst pair needs the memory array to show correct hrdata on the waveform.

so changing the APB model and SRAM model with the memory array will solve the problems ryt?

Yes! Exactly. Just replace those two models and everything else stays the same.

What Changes

1. Add declarations in architecture:

```
vhdl
```

```
-- APB memory
type apb_mem_array is array(0 to 255)
  of std_logic_vector(31 downto 0);
signal apb_mem : apb_mem_array := (others => (others => '0'));

-- SRAM memory
type sram_array is array(0 to 1023)
  of std_logic_vector(31 downto 0);
signal sram_mem : sram_array := (others => (others => '0'));
```

2. Replace APB slave model process

3. Replace SRAM model process

What Gets Fixed

Problem	Fixed?
P2 Read returns 0 instead of 0xDEADBEEF	✓
B2 hrdata=0 instead of written value	✓
All INCR/WRAP write-then-read pairs showing hrdata=0	✓
BUSY tests	Already fine, unaffected
Error tests	Already fine, unaffected

What Stays The Same

- AHB master model — no change
- Stimulus process — no change
- DUT instantiation — no change
- Clock generation — no change
- All signal declarations — no change

So yes, just those two model replacements fix all the data correctness issues.

give me the ready to paste part for this slave and sram model apart alone..
i don't know where to add the architecture declaration

Here's the ready-to-paste code.

Step 1: Architecture Declarations

Paste this **right after** `architecture sim of tb_bridge_top is` and **before** `begin`:

```
vhdl

-- APB memory array
type apb_mem_array is array(0 to 255)
  of std_logic_vector(31 downto 0);
signal apb_mem : apb_mem_array := (others => (others => '0'));

-- SRAM memory array
type sram_array is array(0 to 1023)
  of std_logic_vector(31 downto 0);
signal sram_mem : sram_array := (others => (others => '0'));
```

Step 2: Replace APB Slave Model

Find and replace the entire `apb_slave_model` process with this:

```
vhdl

apb_slave_model : process(clk)
begin
  if rising_edge(clk) then
    if psel = '1' and penable = '1' then
      if pwrite = '1' then
        -- WRITE: store pdata into apb_mem
        apb_mem(to_integer(
          unsigned(paddr(9 downto 2)))) <= pdata;
        prdata <= (others => '0');
      else
        -- READ: return stored value
        prdata <= apb_mem(to_integer(
          unsigned(paddr(9 downto 2))));
      end if;
    else
      prdata <= (others => '0');
    end if;
  end if;
end process;
```

Step 3: Replace SRAM Model

Find and replace the entire `sram_model` process with this:

```
vhdl

sram_model : process(clk)
begin
    if rising_edge(clk) then
        if sram_ce1 = '0' then
            if sram_wen = '0' then
                -- WRITE: store sram_wdata into sram_mem
                sram_mem(to_integer(
                    unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
                sram_rdata <= (others => '0');
            elsif sram_oen = '0' then
                -- READ: return stored value
                if unsigned(sram_addr) >= 16#F00000# then
                    -- Out-of-range: return error pattern
                    sram_rdata <= x"DEAD0000";
                else
                    sram_rdata <= sram_mem(to_integer(
                        unsigned(sram_addr(11 downto 2))));
                end if;
            else
                sram_rdata <= (others => '0');
            end if;
        else
            sram_rdata <= (others => '0');
        end if;
    end if;
end process;
```

Where Exactly in the File

```
architecture sim of tb_bridge_top is

    signal clk : ...           <- existing signals
    signal rst : ...
    ...
    signal irq : ...
```

↓ PASTE STEP 1 HERE ↓

```

type apb_mem_array ...
signal apb_mem ...
type sram_array ...
signal sram_mem ...

component eth_module ... ← existing component

begin

    DUT : eth_module ...      ← existing DUT

    clk_gen : process ...     ← existing clock

    ↓ REPLACE WITH STEP 2 ↓
    apb_slave_model : process ...

    ↓ REPLACE WITH STEP 3 ↓
    sram_model : process ...

    ahb_master_model : ...    ← existing, no change
    stimulus : ...           ← existing, no change

end architecture sim;

```

is the sram_wen = 0 checking if it is writing?

Yes!

In your DUT `mem_in_type`:

```

vhdl

mem_wen : std_logic;  -- active LOW

```

So:

- `sram_wen = '0'` → **WRITE** (signal is asserted, active low)
- `sram_wen = '1'` → **READ** (signal is deasserted)

So in the SRAM model:

```

vhdl

```

```
if sram_wen = '0' then
  -- WRITE: store sram_wdata    <- correct
elsif sram_oen = '0' then
  -- READ: return stored value  <- correct
end if;
```

This is correct. But earlier in the **old combinational SRAM model** I had:

```
vhdl

if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
```

That `sram_wen = '1'` was checking for **not writing** (i.e. read condition) — which is what you caught earlier as a potential bug.

So to summarize:

Signal value	Meaning
<code>sram_wen = '0'</code>	Writing (active low asserted)
<code>sram_wen = '1'</code>	Not writing / Reading

```
I think they are the opposite in my ahb_mem (because the first code u
gave me's sram model  sram_model : process(sram_ce1, sram_oen,
sram_addr)
begin
  if sram_ce1 = '0' and sram_oen = '0' then
    -- Return address-dependent read data so we can verify
    sram_rdata <= std_logic_vector(
```

Show more

Good catch! The first working SRAM model didn't even check `sram_wen` at all — it only checked `ce1` and `oen`. So based on what worked, here is the corrected SRAM model:

Replace SRAM Model With This:

```
vhdl
```

```

sram_model : process(clk)
begin
  if rising_edge(clk) then
    if sram_ce1 = '0' then
      if sram_oen = '1' then
        -- WRITE: oen=1 means not reading, so we are writing
        sram_mem(to_integer(
          unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
        sram_rdata <= (others => '0');
      elsif sram_oen = '0' then
        -- READ: oen=0 means output enabled, so we are reading
        if unsigned(sram_addr) >= 16#F00000# then
          sram_rdata <= x"DEAD0000";
        else
          sram_rdata <= sram_mem(to_integer(
            unsigned(sram_addr(11 downto 2))));
        end if;
      end if;
    else
      sram_rdata <= (others => '0');
    end if;
  end if;
end process;

```

Logic Explanation

Based on your working first code:

- `ce1=0` → chip is selected
- `oen=0` → output enabled → **READ**
- `oen=1` → output disabled → **WRITE**

`sram_wen` is not used as the distinguishing signal — `sram_oen` is what differentiates read from write in your DUT.

if suppose they do use wen as well..and the logic is 1- write and 0- write...how would the model look like..give me the code

Here is the SRAM model if `sram_wen=1` means write and `sram_wen=0` means read:

```
vhdl

sram_model : process(clk)
begin
  if rising_edge(clk) then
    if sram_ce1 = '0' then
      if sram_wen = '1' then
        -- WRITE: wen=1 means write
        sram_mem(to_integer(
          unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
        sram_rdata <= (others => '0');
      elsif sram_wen = '0' and sram_oen = '0' then
        -- READ: wen=0 and oen=0 means read
        if unsigned(sram_addr) >= 16#F00000# then
          sram_rdata <= x"DEAD0000";
        else
          sram_rdata <= sram_mem(to_integer(
            unsigned(sram_addr(11 downto 2))));
        end if;
      else
        sram_rdata <= (others => '0');
      end if;
    else
      sram_rdata <= (others => '0');
    end if;
  end if;
end process;
```

Comparison of All 3 Versions

Version	Write condition	Read condition
Your working model	ce1=0, oen=1	ce1=0, oen=0
Active low wen	ce1=0, wen=0	ce1=0, wen=1, oen=0
Active high wen (this one)	ce1=0, wen=1	ce1=0, wen=0, oen=0



